

Predicting Steering Vectors and Adapter Weights for Few-Shot Author-Style Transfer

Leonard Popp

Danni Liu

Supriti Sinhamahapatra

Jan Niehues

Karlsruhe Institute of Technology, Germany

leonard.popp@student.kit.edu, {firstname.lastname}@kit.edu

Abstract

Adapting large language models to an individual author’s style from a few examples is challenging, and scientific writing sharpens the difficulty: formal conventions leave little surface variation, and authors write about their own topics, so extracted “style” easily entangles with content. We study style-conditioned abstract generation from a few example abstracts per author and propose three methods: (1) contrastive activation steering, (2) a network that predicts steering vectors, and (3) a hypernetwork that predicts LoRA adapters. We find a consistent trade-off between style imitation and output quality: fine-tuning buys most of the available style signal but forfeits fluency, while the hypernetwork achieves the best trade-off on both seen and unseen authors. Our steering operates at author level, contrasting an author’s abstracts against style-neutral generations for the same content. This holds topic fixed, removes the need for a predefined style inventory, and outperforms inventory-based steering. Moreover, our analyses demonstrate that manually extracted and predicted steering vectors are near-orthogonal yet score comparably, indicating that style conditioning here can admit at least two unrelated directions rather than requiring one particular axis.

1 Introduction

By in-context learning (Brown et al., 2020), large language models can solve a wide range of tasks by conditioning on a few demonstrations without parameter updates. Author-level style control, however, is not reliably one of these tasks. While coarse style attributes such as sentiment or emotion can be controlled through activation-level interventions (Subramani et al., 2022; Rinsky et al., 2024; Koenen et al., 2024; Zhang et al., 2025b), imitating an individual author from a handful of texts is harder: Horvitz et al. (2024) find that an 800M model fine-tuned on authorship embeddings outperforms GPT-4 at few-shot transfer toward specific users. This is

the *cold-start* setting of user-level personalization (Zhang et al., 2025c): per-user fine-tuning (Tan et al., 2024c; Dan et al., 2025) requires a history the new author does not have, retrieval over that history is equally unavailable (Salemi et al., 2024; Richardson et al., 2023), and a few examples are often insufficient for the model to extract style (Reif et al., 2022; Patel et al., 2022). Scientific writing exacerbates the difficulty. Formal conventions leave little surface variation, and authors write in their style about their topics, so a model extracting an author’s style risks copying content rather than form (Wegmann et al., 2022; Patel et al., 2025).

Existing approaches address parts of this problem but leave a clear gap. Activation steering has been evaluated on coarse stylistic distinctions (Koenen et al., 2024) or in settings with substantial per-attribute supervision (Rinsky et al., 2024; Tan et al., 2024a), and STYLEVECTOR (Zhang et al., 2025b) computes and stores one contrastive vector per user offline. Adapters generated by hypernetworks (Ha et al., 2016) remove per-user training, but Text-to-LoRA (Charakorn et al., 2025) conditions on natural-language task descriptions, and P2P (Tan et al., 2026) on user profiles assembled from history summaries and retrieval. None of these works targets fine-grained, author-level style in a strict few-shot regime. We study style-conditioned abstract generation from three example abstracts per author, comparing three baselines (zero-shot, few-shot, LoRA) against three proposed methods (§4): 1) contrastive activation steering, 2) a network predicting steering vectors, and 3) a hypernetwork predicting LoRA adapters. Our contributions are:

- We expose a consistent trade-off between style imitation and output quality across all six methods, on which the hypernetwork appears to occupy the most efficient position (§6.4).
- We show that manually computed and predicted steering vectors are near-orthogonal yet yield

comparable style accuracy, so at least two unrelated directions support style conditioning and no particular axis is required (§7.1).

- We adapt contrastive steering to the author level by contrasting ground-truth abstracts against style-neutral model generations for the same content, and find it matches or exceeds the method of [Konen et al. \(2024\)](#) (§7.2).

2 Related Work

Few-Shot Personalization of LLMs Style-conditioned generation sits at the intersection of text style transfer and user-level personalization ([Zhang et al., 2025c](#)). [Horvitz et al. \(2024\)](#) target the low-data regime by conditioning a small model on authorship embeddings for few-shot transfer toward individual users. The strongest personalization results, however, come from per-user fine-tuning that trains dedicated LoRA adapters ([Hu et al., 2022](#)) for each user ([Tan et al., 2024c,b](#); [Dan et al., 2025](#)). This scales poorly in the number of users and degrades when user histories are limited. This work targets the cold-start setting at author-level granularity, in scientific writing, a domain whose formal constraints leave comparatively little surface variation to exploit.

Activation Steering Hidden activations contain latent directions that control stylistic attributes ([Subramani et al., 2022](#); [Zhang et al., 2025a](#)). [Konen et al. \(2024\)](#) derive style vectors by contrasting mean activations of a target style against a fixed set of remaining styles, at a coarse granularity such as modern versus old English. We instead contrast an author’s ground-truth abstracts against style-neutral abstracts the model generates for the same content, which removes the need for a predefined inventory, holds topic constant, and operates at author level. The matched-content contrast is close in spirit to [Zhang et al. \(2025b\)](#); our contribution is its use at author level in scientific writing and, primarily, the prediction of author-specific interventions from three examples. [Zhang et al. \(2025b\)](#) store one contrastive vector per user, computed offline from that user’s texts. We further train a network to predict steering vectors in a single forward pass from three examples, so no per-user artifact is computed or stored.

Hypernetwork-Generated Adapters Hypernetworks ([Ha et al., 2016](#)) generate adaptation parameters directly, removing the need for per-user

training: prior work produces attribute-conditioned prefixes for a frozen LM ([Clive et al., 2022](#)) and user-specific LoRA adapters for perspectivist classification ([Ignatev et al., 2025](#)). Text-to-LoRA ([Charakorn et al., 2025](#)), whose backbone we adopt, maps a natural-language task description to LoRA weights for zero-shot adaptation to unseen tasks. We instead predict style-conditioned adapters for a fixed task from an aggregated embedding of examples. Closest to our setting, [Tan et al. \(2026\)](#) map an encoded user profile to LoRA weights for long-form generation, but rely on an LLM-generated history summary and a retrieval pipeline, whereas we condition on three abstracts alone.

3 Task and Evaluation

Task Formulation Given a target author a , a reference set $R_a = \{r_1, r_2, r_3\}$ of three abstracts by a , and a *structured outline* s of an unseen paper, the task is to generate an abstract $\hat{y} = M(s, R_a)$ that describes s in the style of a . Outlines are produced from the paper’s L^AT_EX source (§5.1); no model observes the original abstract as input. The base model M is frozen throughout, and no method requires per-author training at deployment. The small size of R_a is a property of the cold-start setting ([Zhang et al., 2025c](#)) rather than a hyperparameter. Unlike prior work, we assume neither a rich user profile nor a retrieval corpus ([Tan et al., 2026](#)). We evaluate on authors seen during training and on held-out authors entirely (§5.1).

Author Style Evaluation We measure style by authorship attribution. Instead of a standard classifier which fixes the label set at training time, our unseen-author regime requires attributing text to new authors introduced at inference from three examples. Therefore, we adopt k -NN retrieval in a learned embedding space, which admits new authors without retraining. Specifically, an encoder is first fine-tuned on authorship classification. Then we discard its head, and create document embeddings by mean-pooling. With the example abstracts forming the reference database, a query is assigned the majority label among its k nearest neighbors under cosine distance, falling back to the nearest neighbor when no majority exists. We report accuracy and mean reciprocal rank (MRR), the latter being continuous and therefore less sensitive to sampling variance. Note that style and content are not fully separable in this metric, as abstracts generated without author information are attributed well

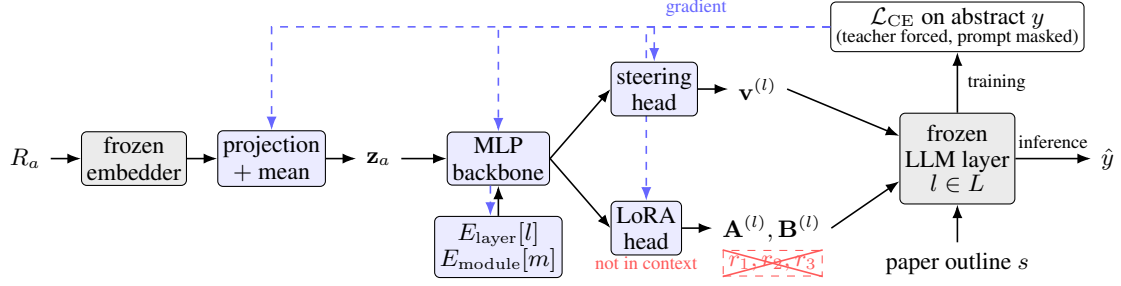


Figure 1: Both style control networks differ only in their output head. The author-specific example abstracts R_a are embedded into z_a , concatenated with a layer embedding (for the hypernetwork, also a module embedding) and passed through the MLP backbone. The steering head emits a vector added to the residual stream. The LoRA head emits adapter weights inserted into the attention projections. The network is queried once per target layer $l \in L$ (and per module m for LoRA prediction).

above chance (§6.1), a known issue of authorship representations (Wegmann et al., 2022; Huertas-Tato et al., 2024). So we only use style accuracy to compare models rather than as an absolute metric.

Topic Preservation Evaluation The generated abstracts must not deviate from the main text’s topic. Therefore, we detect off-topic generation with a binary LLM-judge. GPT-OSS-20B (OpenAI, 2025) receives an abstract and the introduction of a paper and decides whether both originate from the same document (prompt in Appendix A.1).

Text Quality Evaluation Besides topic preservation, we evaluate the intrinsic quality of the generated abstracts. Perplexity is an established metric for text fluency (Holtzman et al., 2020) but is untrustworthy here, as it rewards the predictable, repetitive text that aggressive style conditioning may produce. Instead, we measure quality by pairwise preference by letting GPT-OSS-20B compare each generated abstract against the author’s real abstract for the same paper (prompt in Appendix A.2). Each pair is judged twice in swapped order to cancel position bias (Ko et al., 2020; Qin et al., 2024). We report the preference rate against the human-written abstract. Since our corpus predates 2022 when LLM-assisted writing emerged (§5.1) and the judge tends to favor LLM-generated content (Zheng et al., 2023), preference rates are high across all models. However, note that it is the differences between the preference rates, not their absolute level, that carry the information.

4 Style Control Methods

Manual Contrastive Steering Hidden activations contain directions that control stylistic attributes, and adding a style vector to selected layers

steers generation toward the corresponding style. Prior work (Konen et al., 2024) obtains the steering vector by contrasting a target style against the mean activation of all other styles in a fixed inventory. This does not transfer to our case, as such an inventory hardly exists in practice, and contrasting one author against others writing about different topics leaves content in the vector. Instead, we contrast each reference abstract against the model’s own “neutral” abstract for the same paper, generated zero-shot from s_i in the model’s default style. Specifically, for layer l , the steering vector is

$$\mathbf{v}_a^{(l)} = \bar{\mathbf{a}}_{\text{auth}}^{(l)} - \bar{\mathbf{a}}_{\text{gen}}^{(l)}, \quad (1)$$

where both terms are activations mean-aggregated over tokens. The three per-reference vectors are averaged and injected during generation,

$$\mathbf{a}_{\text{steered}}^{(l)}(x) = \mathbf{a}^{(l)}(x) + \lambda \mathbf{v}_a^{(l)}, \quad (2)$$

with intensity λ and target layers L selected on validation (§5.3). Our approach outperforms the inventory-based method of Konen et al. (2024), as shown in §7.2. Three constraints remain. First, the contrast requires one neutral generation per reference, which in turn requires the outline of the corresponding reference paper. Manual steering therefore consumes strictly more than R_a and does not satisfy the deployment setting of §3; we report it not as a competing method under that setting, but as a reference point the two learned methods must match using the three abstracts alone. Second, it requires a λ search per setting: we sweep $\lambda \in \langle \text{grid} \rangle$ on validation and select $\lambda = 0.875$ for the test configuration, whereas the layer comparison in Table 5 was run at $\lambda = 1.0$ before this search. Third, the intervention is applied to the residual stream

at the ⟨output of each target block / post-attention position⟩ during both prefill and autoregressive decoding, and the activations in Equation 1 are mean-pooled over the ⟨abstract tokens only, with prompt tokens excluded⟩.

Learned Steering We tackle both constraints by predicting the intervention directly from R_a . Each reference is embedded with a frozen STAR encoder¹ (Huertas-Tato et al., 2024), then projected and averaged into an author representation

$$\mathbf{z}_a = \frac{1}{|R_a|} \sum_i \text{Proj}(\text{encoder}(r_i)). \quad (3)$$

Concatenating $\mathbf{z}_a$ with a learned layer embedding forms the input passed to an MLP backbone with residual blocks following Charakorn et al. (2025) (Figure 1; details in Appendix B). A linear head projects to the hidden dimension of M and its output is added to the residual stream, $\mathbf{a}_{\text{steered}}^{(l)}(x) = \mathbf{a}^{(l)}(x) + \mathbf{v}_a^{(l)}$. The head is zero-initialized, so the intervention is a no-op at the start of training. No explicit λ is applied, as the network learns per-layer magnitudes. Despite this flexibility, a downside is that a single vector per layer shifts the residual stream uniformly for every token, independently of what is being written. We tackle this next.

Hypernetwork for LoRA Weights Unlike steering vectors, LoRA adapters condition their update on the layer input x rather than adding a constant offset, so the intervention varies with what the model is writing. Therefore, we extend the previous setup to predict LoRA weights, inspired by Charakorn et al. (2025) which predicts LoRA weights given novel task descriptions. Here, we adopt their architecture, with a module embedding for $m \in \{\text{q_proj}, \text{v_proj}\}$ concatenated alongside the layer embedding, and four linear heads emit the flattened adapters $\mathbf{A}^{(l)} \in \mathbb{R}^{r \times d_{\text{in}}}$ and $\mathbf{B}^{(l)} \in \mathbb{R}^{d_{\text{out}} \times r}$, where r is the LoRA rank. This modifies the forward pass as

$$\mathbf{a}^{(l)}(x) = \mathbf{W}_0^{(l)} x + \mathbf{B}^{(l)} \mathbf{A}^{(l)} x. \quad (4)$$

The module initialization mirrors standard LoRA, where the $\mathbf{A}$ -heads are initialized with zero weights and uniform biases, and the $\mathbf{B}$ -heads with zero weights and biases. This leaves the adapters as a no-op at initialization and was essential for training

¹STAR is trained contrastively for authorship and therefore expected to be more stylistic than semantic. It also outperformed Qwen/Qwen3-Embedding in initial experiments.

stability in our case. We treat the set of adapted layers as a design choice, restricting it to a middle-layer subset as it acts as a regularizer and improves downstream metrics (Table 5).

Training As shown in Figure 1, both learned steering and the hypernetworks are trained end-to-end with the author’s ground-truth abstract. The base model and the encoder stay frozen, and gradients pass through M into the prediction network alone. Crucially, M receives the zero-shot prompt: the references never enter its context, so style is carried entirely by the predicted parameters.

5 Experimental Setup

5.1 Data Creation

Source and Filtering Our dataset is built from an arXiv snapshot. We retain papers with permissive licenses² in the cs.AI and cs.LG categories. Publication year is capped at 2022, so that ground-truth abstracts cannot have been written by a modern LLM: the style targets are *guaranteed human*. Lacking unique author identifiers, we treat identical author names as one person and discard authors publishing in over five arXiv categories as a disambiguation heuristic. We assume the first author is a paper’s primary writer. Papers without parsable L^AT_EX source or an introduction are excluded.

Outline Creation As defined in §3, the models receive an outline rather than the full paper. This ensures that minimal style information is available in the input. We clean the L^AT_EX source to plain text and prompt Qwen3-4B-Instruct-2507 to produce bullet points under four headings: Context, Methodology, Results, and Conclusion. Additional details and prompts are in Appendix A.3.

Statistics After filtering, the dataset contains 1,232 papers from 203 authors, averaging 6.07 papers each (range 5-14). One paper per author forms the validation set and one the test set, and the remaining 826 form the training set. Each entry pairs a paper’s outline with its ground-truth abstract, its introduction, and three abstracts sampled randomly from the same author’s remaining papers.

Unseen Author Test Set To evaluate generalization, we create a second test set where none of the authors appeared in training for either the generation models or the author classifier. In this set, the

²Some licenses require mentioning the authors. We include this in our repository (deanonymized upon publication).

authors have exactly four papers, which guarantees disjointness from the base dataset. It contains 1,312 papers from 328 authors: three per author serve as prompt examples and one as the test sample.

5.2 Baselines

We compare our methods to three standard ones, with the corresponding prompts in Appendix A.4.

Zero-Shot The base model generates an abstract from the outline alone. It receives no author information, and therefore provides a lower bound on style accuracy and a quality anchor.

Few-Shot The zero-shot prompt is extended with the three reference abstracts, and the model is instructed to extract and apply the author’s style. This measures style transfer without parameter updates.

LoRA The base model is fine-tuned on the few-shot prompt with adapters on the query and value projections at rank $r = 8$, matching the hypernetwork’s adapter budget so that the two are directly comparable. It represents the strongest style baseline. Note that here LoRA is *not per-author*, as our data only contain roughly five abstracts per author. Instead, a single set of LoRA adapters is optimized across all training authors and conditioned at inference on the three author-specific examples.

5.3 Implementation Details

All models use Qwen3-4B-Instruct-2507 (Qwen Team, 2025) as the frozen base model M (36 layers, $d = 2,560$). We decode with sampling ($T = 0.7$, repetition penalty 1.05, top- $p = 0.8$, top- $k = 20$) and generate 1,024 tokens at most. The k NN author classifier is a fine-tuned distilbert-base-uncased with $k = 3$ and cosine distance. Both learned steering and the hypernetwork use a four-block MLP backbone with an author representation of 128 dimensions.

Configuration searches use the validation set, while the test set is reserved for final evaluation. Manual steering uses our contrast method at layers 18-20 with $\lambda = 0.875$. Learned steering encodes the three example abstracts with the STAR encoder (Huertas-Tato et al., 2024) and predicts a shift vector for all 36 layers. The hypernetwork uses the same STAR encoder and predicts rank-8 adapters for the query and value projections, matching the LoRA baseline’s rank. Further implementation and hyperparameter details are in Appendix B.

	Pred. Positive	Pred. Negative	Total
GT Positive	1,216	16	1,232
GT Negative	2	1,230	1,232
Total	1,218	1,246	2,464

Table 1: Confusion matrix for the topic preservation judge. It attains 99.3% accuracy, misclassifying 18 of 2,464 abstract-introduction pairs. Negative pairs are constructed by randomly pairing abstracts with unrelated introductions.

	Content	Style	Style Acc. (%)
<i>Target author’s papers</i>			
(1)	target	human (ground truth)	53.2
(2)	target	LoRA (target author)	49.8
(3)	target	none (zero-shot)	31.0
<i>Different author’s papers</i>			
(4)	foreign	LoRA (target author)	0.3
(5)	foreign	none (zero-shot)	0.0
(6)	Random guessing		0.5

Table 2: Evaluation of the k -NN author classifier. Each row is a set of abstracts labeled with the target author, varying the paper (*Content*) and the source of writing style (*Style*). Removing style at fixed content (row 3) costs 22.2 points, yet the remaining 31.0% is attainable from content alone. Conversely, styling a different author’s paper (rows 4-5) falls to chance.

6 Quantitative Results

6.1 Metric Validation

Before reporting the main results, we validate the two automatic metrics: the topic preservation judge and the author classifier.

Topic Preservation Judge We first verify that GPT-OSS-20B can reliably detect whether a generated abstract describes the intended paper. As shown in Table 1, on an evaluation set of 2,464 abstract-introduction pairs, the model achieves 99.3% accuracy, with 16 false negatives and 2 false positives. Therefore, we adopt it as a guard against off-topic generation.

Content Entangled with Style Table 2 shows results of evaluating the k -NN style classifier. First, the classifier attains 53.2% accuracy on held-out ground-truth abstracts against random guess accuracy of 0.5%. Two experiments establish what it measures. Removing style while holding content fixed, by classifying style-neutral abstracts generated from the same outlines, drops accuracy to 31.0%. The 22.2-point difference is what the classifier loses when author style is removed under this

intervention, and the remaining 31.0 is what content alone supplies; neither figure isolates a purely stylistic or purely content-driven component. This score stays far above chance, and as these abstracts preserve the paper’s topic, the residual shows the classifier also draws on content, an entanglement consistent with known difficulties in authorship attribution (Huertas-Tato et al., 2024; Wegmann et al., 2022). Reversing the manipulation, by varying content while holding style conditioning fixed, sends every model to or below chance, equally consistent with a classifier that cannot recognize unfamiliar content and with generators that cannot carry style across topics.

Interpreting Style Accuracy The 53.2% ground-truth level reflects the classifier’s attribution ability rather than a property of the generative models. More consequentially, the style-neutral condition above comes from the same zero-shot model that serves as our first baseline in §6.2, so its 31.0% is not a style score but the level attainable from content alone. We therefore read style accuracy relative to a content-only *reference point* of 31.0% and a classifier *ceiling* of 53.2%. Neither is a strict bound: the few-shot baseline falls below 31.0% (§6.2).

6.2 Results on Seen Authors

Table 3a reports test set results for our three approaches alongside the three baselines and the ground truth, evaluated on the 203 authors seen during training. We report preference rate as the primary quality metric and author classifier accuracy as the primary style metric. All models preserve the paper’s topic at comparable levels (95.6-97.5%), which is a precondition for comparing style accuracy rather than a discriminating result.

Few-shot baseline fails to convey author style. The few-shot baseline scores below the zero-shot baseline on both style metrics (28.1% and 0.36 against 31.0% and 0.40), despite receiving the same examples the LoRA baseline is trained with. It thus falls below the content-only reference point, potentially because instruction-following pulls the output toward a generic instructed register, moving embeddings away from any specific author. Quality degrades as well (91.5% preference rate).

Direct fine-tuning maximizes style at the cost of quality. The LoRA setup attains the highest style accuracy at 49.8%, roughly 84% of the range

Model	Pref. Rate (%)	Topic Pres. (%)	Style Acc. (%)	MRR
Zero-shot	97.9	96.6	31.0	0.40
Few-shot	91.5	97.0	28.1	0.36
LoRA	38.1	95.6	49.8	0.55
Manual steering	94.3	97.5	34.5	0.42
Learned steering	89.6	96.1	35.0	0.43
Hypernet LoRA	91.3	97.0	<u>36.5</u>	<u>0.46</u>
Ground truth	–	97.0	53.2	0.61

(a) Seen authors test set ($n = 203$). Style accuracy is bounded below by 31.0, the level attainable from content alone (§6.1), and above by the classifier ceiling of 53.2.

Model	Pref. Rate (%)	Topic Pres. (%)	Style Acc. (%)	MRR
Zero-shot	97.4	99.1	22.9	0.31
Few-shot	90.2	99.7	18.3	0.26
LoRA	39.0	97.0	37.5	0.47
Manual steering	95.1	98.5	26.2	0.37
Learned steering	87.5	99.4	28.4	0.38
Hypernet LoRA	<u>95.5</u>	99.4	<u>31.4</u>	<u>0.40</u>
Ground truth	–	100.0	41.5	0.49

(b) Unseen authors test set ($n = 328$). Neither the models nor the author classifier were trained on any sample from this set. Style accuracy is bounded below by 22.9, and above by the classifier ceiling of 41.5.

Table 3: Style and quality results on two tests sets with (a) seen and (b) unseen authors.

above the content-only reference point. However, its preference rate falls drastically to 38.1% from 97.9% for the zero-shot baseline, so fine-tuning purchases nearly all of the available style signal by sacrificing what makes the abstracts usable. Manual inspection reveals fluency issues with frequent repetitions.

Activation steering preserves quality but yields small gains. Both steering approaches improve style while leaving quality largely intact. Manual steering reaches 34.5% at a preference rate of 94.3%, the highest of any model improving on the zero-shot style accuracy, and the prediction network reaches 35.0% at 89.6%. Predicting vectors therefore matches hand-computed ones on style without a neutral generation per reference or a search over intensity. The two differ by a single abstract, and §7.1 shows they achieve this by steering in nearly orthogonal directions.

Hypernet LoRA balances style and quality. The hypernetwork LoRA reaches 36.5%, roughly a quarter of the range, while retaining a preference rate of 91.3% that direct fine-tuning forfeits. Both amortize across authors rather than fitting each sep-

arately, differing in that LoRA carries style through the context window with a shared adapter, whereas the hypernetwork carries it through adapter weights predicted per author. Note the two methods are matched on adapter budget but not on information: LoRA reads the example text directly and can copy surface phrasing, while the hypernetwork conditions only on their embeddings.

6.3 Generalization to Unseen Authors

So far, the models were tested on authors present in the training data. We now assess if style conditioning transfers to authors that neither the models nor the author classifier have seen (Table 3b).

Model rankings are preserved. Every value is lower than on the seen authors, including the ground truth (41.5% against 53.2%). Only the relative ordering is informative, and it is identical, with few-shot again last and below the zero-shot baseline. As the seen and unseen sets share no authors, generation noise is an implausible explanation for the few-shot result.

Hypernet retains position. The hypernetwork improves style by 8.5 points over the zero-shot baseline while giving up 1.9 points of preference rate, ranking second on all three metrics, and the standard LoRA setup again buys the highest style accuracy at a preference rate of 39.0%. The trade-off structure thus reappears unchanged on authors no component of the system was trained on. The hypernetwork now recovers close to half of the range against roughly 79% for LoRA, where the seen-author fractions were a quarter and 84%, though the narrower unseen range inflates these fractions and they are not directly comparable across sets.

6.4 Style-Quality Trade-Off

Figure 2 plots preference rate against style accuracy. We characterize the trade-off by Pareto dominance on point estimates. Three models are Pareto-optimal: the zero-shot baseline on preference rate, LoRA on style accuracy, and the hypernetwork LoRA between them. On the seen authors the frontier also contains manual steering, so the hypernetwork is the only one of our approaches Pareto-optimal on *both* sets at the configurations of Table 3a. Two qualifications apply. First, the two learned methods are reported at different intervention scopes: learned steering over all 36 layers and the hypernetwork over layers 18–20. At the common scope 18–20 (Table 5), the hypernetwork

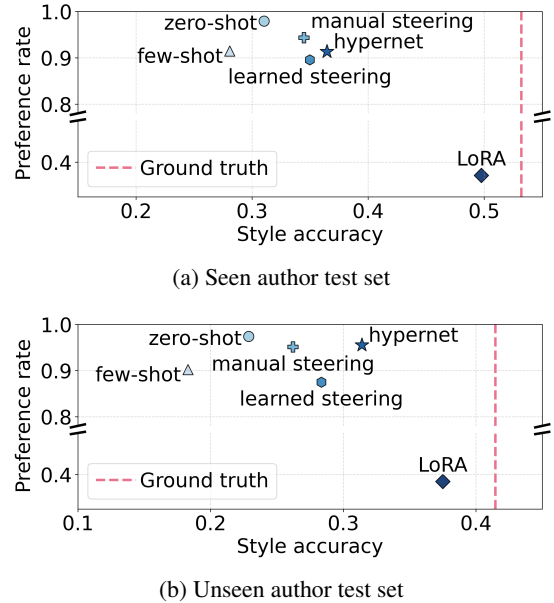


Figure 2: Style-quality trade-off by plotting the preference rate against style accuracy for each method, on seen (a) and unseen authors (b). The hypernetwork sits immediately before quality degrades.

retains higher style accuracy (36.5% vs. 34.0%) but learned steering attains the higher preference rate (97.5% vs. 91.3%), so neither dominates and the seen-set frontier is scope-dependent. Second, several style-accuracy differences correspond to few examples; manual and predicted steering differ by a single abstract on the seen-author test set. We therefore report paired bootstrap 95% confidence intervals in Table 3a and read Pareto statements as descriptive of point estimates rather than as significance claims. Relative to the hypernetwork, on the unseen set, LoRA buys 6.1 points of style accuracy for 56.5 points of preference rate. In contrast, the hypernetwork occupies the frontier between the two, retaining quality while recovering a meaningful share of the style range.

7 Analyses

In addition to quantitative results above, we conduct several analyses.

7.1 No Single Style Axis

As shown previously in Table 3a, manual steering and the activation vector prediction network reach comparable style accuracies. Since both add a vector to the model’s hidden activations, we ask: do both methods achieve the same *behavior* with the same *internal mechanisms*? Table 4 compares the vectors both produce at layers 18 to 20, averaged

Layer	L2 Norm			L1 / L2			Cosine Similarity
	Manual	Pred.	Hidden	Manual	Pred.	Hidden	Manual vs. Pred.
18	13.4	14.3	39.9	17.3	40.1	11.8	−0.03
19	13.6	8.2	46.4	18.1	39.5	12.1	−0.05
20	15.1	12.2	49.4	19.4	39.7	12.0	−0.04

Table 4: Per-layer statistics for manual and predicted steering vectors, averaged over the 203 validation samples. *Hidden* indicates the *unsteered* activations the vectors are added to. L1/L2 measures how broadly a vector spreads its mass, with 40.4 expected of a dense Gaussian vector in 2,560 dimensions and 50.6 of a uniform one. The final column reports the average cosine similarity between the manual and predicted vector at each layer.

over 203 validation samples.

Manual and learned steering vectors are near-orthogonal. As shown in the last column of Table 4, the average cosine similarity between the manual and the predicted vectors is near zero at every layer (−0.03, −0.05, −0.04). In 2,560 dimensions this is the value expected of unrelated vectors, suggesting that the two sets of vectors are therefore *nearly orthogonal*. The near-orthogonality combined with their equivalent style accuracy shows that the predictor does not recover the manually extracted direction, and that at least two unrelated directions reach comparable style accuracy. This does not rule out the existence of a privileged direction; it shows that reaching one is not necessary. Establishing the stronger claim would require comparing vectors extracted from disjoint reference subsets of the same author against vectors from different authors, which we leave to future work.

Manual and learned steering vectors have similar magnitudes but different density. Across the two methods, the magnitude of the resulting perturbation is comparable. As shown in Table 4, both produce vectors whose L2 norm falls between 18% and 36% of the corresponding hidden activation norm, which suggests that a similar quantity of perturbation is required to influence the model’s output regardless of the direction chosen. Despite comparable magnitudes, their *internal structure* does not match. The “L1/L2” column in Table 4 measures how broadly a vector spreads its mass. For the predicted vectors it sits at 39.5 to 40.1 at every layer, matching the value expected of a dense Gaussian vector in 2,560 dimensions. In contrast, the manual vectors reach only 17.3 to 19.4. The predicted vectors are therefore *dense*, spreading a small perturbation across many dimensions, whereas the manual vectors are *sparse*, concentrating the perturbation in a few dominant ones.

Layers	Style Acc. (%)	MRR	Pref. Rate (%)
<i>Manual steering (validation)</i>			
18–20, ours	36.0	–	–
20–22, ours	35.0	–	–
all 36, ours	2.5	–	–
18–20, Konen et al. (2024)	34.0	–	–
20–22, Konen et al. (2024)	31.0	–	–
<i>Learned steering (test)</i>			
18–20	34.0	0.44	97.5
all 36	35.0	0.43	89.6
<i>Hypernetwork LoRA (test)</i>			
18–20	36.5	0.46	91.3
all 36	34.5	0.43	71.4

Table 5: Effect of layer selection. Manual steering is reported on the validation set at $\lambda = 1.0$, where the configuration search measured style accuracy only (hence “–” for MRR and Pref. Rate). The learned interventions are reported on the test set.

7.2 Layer Selection

Table 5 shows the impact of which layers of the base model our interventions act on.

Manual steering requires a narrow band. Holding the contrast method fixed, layers 18–20 give the highest style accuracy (36.0%), one point above layers 20–22 (35.0%). The same ordering holds for the contrast of Konen et al. (2024) (34.0% vs. 31.0%), which our construction outperforms at both scopes. Applying the vector to all 36 layers, however, collapses style accuracy to 2.5%, far below the level attainable from content alone (§6.1). A likely explanation is accumulation along the residual stream. At the layers where we measure it, a single style vector displaces the hidden state by roughly a third of its norm (§7.1). Injecting one at every layer therefore compounds the displacement with depth, and risks driving the state out of the distribution the model was trained on. That the surviving band is a middle one matches the findings of Konen et al. (2024), who locate the

same band in a 32-layer model, and evidence that middle layers carry the most abstract representations (Wendler et al., 2024; Liu and Niehues, 2025). Here, mid-depth likely strikes the best balance, as an early perturbation is reshaped by its subsequent layers, while a late one leaves too few layers to realize it in surface form.

Hypernet prefers a narrow band but survives full depth. Compared to predicting adapters for all layers, restricting to layers 18-20 yields higher style accuracy (36.5% vs. 34.5%), MRR (0.46 vs. 0.43), and preference rate (91.3% vs. 71.4%). We evaluated only these two scopes, so the comparison shows that a narrow band beats full depth, not that 18-20 is optimal for the hypernetwork. The failure at full depth is milder than for manual steering. A plausible reason is learning the intervention end-to-end: Manual steering scales every layer by one global λ , while the hypernetwork predicts each layer’s adapter separately, leaving it free to intervene weakly where a layer tolerates less perturbation, as the steering network measurably does (§7.1). That a narrow band wins here suggests this freedom is not sufficient on its own. Restricting the scope likely acts as a regularizer.

Localization is a property of the intervention. Learned steering makes the point more sharply still. At all 36 layers it reaches 35.0% style accuracy at an 89.6% preference rate, the same scope at which the manual vectors collapse to 2.5% on validation. The scope is identical; the vectors are not. Its behavior across scope also differs from the hypernetwork’s: style accuracy is essentially flat (34.0% at layers 18–20 vs. 35.0% at full depth, within a point), while the narrow band instead buys quality, raising the preference rate to 97.5% from 89.6%. Which layers matter, and if narrowing them trades style for quality or improves both, is thus a property of the specific intervention rather than a fixed rule.

8 Conclusion

We studied author-level style conditioning in a strict cold-start setting: three abstracts, no history, no retrieval, and a frozen base model. Our results lead to three practical recommendations. (1) Style conditioning here is not a problem of finding the best intervention, but of navigating a trade-off each intervention pays. (2) The near-orthogonality of manual and learned steering vectors argues against two practices: extraction is not a search for *the* style

direction, since at least two unrelated directions work equally well here and refining the contrast set may only tune an arbitrary choice; and vector similarity cannot validate a method when orthogonal vectors work equally well, only downstream metrics can. (3) In a contrastive steering vector, the negative should carry the style the output must deviate from, here the model’s own default on the same content.

Limitations

Our style metric is entangled with content: style-neutral generations are attributed well above chance, so we can only read accuracy relative to an empirical reference point and ceiling, and comparisons are ordinal rather than absolute. The probe that varies content while fixing style conditioning lands at or below chance, which we cannot attribute to the classifier or to the generators. Readers should therefore treat every style number as a relative ranking within our setup, never as a measure of how well an author is imitated; disentangling the two would require a classifier trained to be content-invariant and a probe that isolates the generator from the metric.

Our main results do not by themselves establish that the learned interventions are author-specific rather than a generic shift toward human-written abstracts. We report the absence of this control as a limitation: without it, the improvement over zero-shot cannot be separated from a generic shift toward human-written abstracts, and the within-versus between-author similarity of predicted vectors and of the effective update $\mathbf{B}^{(l)}\mathbf{A}^{(l)}$ remains unmeasured.

The learned steering and hypernetwork was evaluated at two scopes, so we can only establish that a narrow band beats full depth, not that our selected band is optimal. Our layer choices should be read as a working configuration rather than a tuned one. A full sweep over bands and widths per method is left to future work, and the gap between methods may narrow once each is given its own best scope.

Our methods are run on one base model (Qwen3-4B-Instruct), one broad domain (pre-2022 cs.AI/cs.LG arXiv abstracts), and only English. Whether the trade-off structure is a property of style conditioning or of this configuration remains open. Replication across model scales, families, domains, and other languages is needed before generalizing.

References

- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.
- Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. 2025. [Text-to-LoRA: Instant transformer adaption](#). In *Forty-second International Conference on Machine Learning*.
- Jordan Clive, Kris Cao, and Marek Rei. 2022. [Control prefixes for parameter-efficient text generation](#). In *Proceedings of the Second Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, pages 363–382, Abu Dhabi, United Arab Emirates (Hybrid). Association for Computational Linguistics.
- Yuhao Dan, Jie Zhou, Qin Chen, Junfeng Tian, and Liang He. 2025. [P-react: Synthesizing topic-adaptive reactions of personality traits via mixture of specialized LoRA experts](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6342–6362, Vienna, Austria. Association for Computational Linguistics.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. [The faiss library](#). *IEEE Transactions on Big Data*, 12(2):346–361.
- David Ha, Andrew M. Dai, and Quoc V. Le. 2016. [Hypernetworks](#). *CoRR*, abs/1609.09106.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. [The curious case of neural text de-generation](#). In *International Conference on Learning Representations*.
- Zachary Horvitz, Ajay Patel, Kanishk Singh, Chris Callison-Burch, Kathleen McKeown, and Zhou Yu. 2024. [TinyStyler: Efficient few-shot text style transfer with authorship embeddings](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13376–13390, Miami, Florida, USA. Association for Computational Linguistics.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- Javier Huertas-Tato, Alejandro Martín, and David Camacho. 2024. [Understanding writing style in social media with a supervised contrastively pre-trained transformer](#). *Know.-Based Syst.*, 296(C).
- Daniil Ignatev, Denis Paperno, and Massimo Poesio. 2025. [Hypernetworks for perspectivist adaptation](#). In *Proceedings of the The 4th Workshop on Perspectivist Approaches to NLP*, pages 111–122, Suzhou, China. Association for Computational Linguistics.
- Neel Jain, Ping yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. 2024. [NEFTune: Noisy embeddings improve instruction finetuning](#). In *The Twelfth International Conference on Learning Representations*.
- Miyoung Ko, Jinhyuk Lee, Hyunjae Kim, Gangwoo Kim, and Jaewoo Kang. 2020. [Look at the first sentence: Position bias in question answering](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1109–1121, Online. Association for Computational Linguistics.
- Kai Konen, Sophie Jentzsch, Diaoulé Diallo, Peer Schütt, Oliver Bensch, Roxanne El Baff, Dominik Opitz, and Tobias Hecking. 2024. [Style vectors for steering generative large language models](#). In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 782–802, St. Julian’s, Malta. Association for Computational Linguistics.
- Danni Liu and Jan Niehues. 2025. [Middle-layer representation alignment for cross-lingual transfer in fine-tuned LLMs](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15979–15996, Vienna, Austria. Association for Computational Linguistics.
- OpenAI. 2025. [gpt-oss-120b gpt-oss-20b model card](#). Preprint, arXiv:2508.10925.
- Ajay Patel, Nicholas Andrews, and Chris Callison-Burch. 2022. [Low-resource authorship style transfer with in-context learning](#). *CoRR*, abs/2212.08986.
- Ajay Patel, Jiacheng Zhu, Justin Qiu, Zachary Horvitz, Marianna Apidianaki, Kathleen McKeown, and Chris Callison-Burch. 2025. [StyleDistance: Stronger content-independent style embeddings with synthetic parallel examples](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8662–8685, Albuquerque, New Mexico. Association for Computational Linguistics.
- Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. [Large language models are effective text rankers with pairwise ranking prompting](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1504–1518, Mexico City, Mexico. Association for Computational Linguistics.

- Qwen Team. 2025. [Qwen3 technical report](#). *CoRR*, abs/2505.09388.
- Emily Reif, Daphne Ippolito, Ann Yuan, Andy Coenen, Chris Callison-Burch, and Jason Wei. 2022. [A recipe for arbitrary text style transfer with large language models](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 837–848, Dublin, Ireland. Association for Computational Linguistics.
- Christopher Richardson, Yao Zhang, Kellen Gillespie, Sudipta Kar, Arshdeep Singh, Zeynab Raeesy, Omar Zia Khan, and Abhinav Sethy. 2023. [Integrating summarization and retrieval for enhanced personalization via large language models](#). *CoRR*, abs/2310.20081.
- Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. 2024. [Steering llama 2 via contrastive activation addition](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, Bangkok, Thailand. Association for Computational Linguistics.
- Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2024. [LaMP: When large language models meet personalization](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7370–7392, Bangkok, Thailand. Association for Computational Linguistics.
- Nishant Subramani, Nivedita Suresh, and Matthew Peters. 2022. [Extracting latent steering vectors from pretrained language models](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 566–581, Dublin, Ireland. Association for Computational Linguistics.
- Daniel Tan, David Chanin, Aengus Lynch, Brooks Paige, Dimitrios Kanoulas, Adrià Garriga-Alonso, and Robert Kirk. 2024a. Analysing the generalisation and reliability of steering vectors. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS ’24*, Red Hook, NY, USA. Curran Associates Inc.
- Zhaoxuan Tan, Zheyuan Liu, and Meng Jiang. 2024b. [Personalized pieces: Efficient personalized large language models through collaborative efforts](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 6459–6475, Miami, Florida, USA. Association for Computational Linguistics.
- Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. 2024c. [Democratizing large language models via personalized parameter-efficient fine-tuning](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 6476–6491, Miami, Florida, USA. Association for Computational Linguistics.
- Zhaoxuan Tan, Zixuan Zhang, Haoyang Wen, Zheng Li, Rongzhi Zhang, Pei Chen, Fengran Mo, Zheyuan Liu, Qingkai Zeng, Qingyu Yin, and Meng Jiang. 2026. [Instant personalized large language model adaptation via hypernetwork](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23557–23580, San Diego, California, United States. Association for Computational Linguistics.
- Anna Wegmann, Marijn Schraagen, and Dong Nguyen. 2022. [Same author or just same topic? towards content-independent style representations](#). In *Proceedings of the 7th Workshop on Representation Learning for NLP*, pages 249–268, Dublin, Ireland. Association for Computational Linguistics.
- Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. 2024. [Do llamas work in English? on the latent language of multilingual transformers](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15366–15394, Bangkok, Thailand. Association for Computational Linguistics.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. [Transformers: State-of-the-art natural language processing](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.
- Hanyu Zhang, Xiting Wang, Chengao Li, Xiang Ao, and Qing He. 2025a. [Controlling large language models through concept activation vectors](#). In *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*, pages 25851–25859. AAAI Press.
- Jinghao Zhang, Yuting Liu, Wenjie Wang, Qiang Liu, Shu Wu, Liang Wang, and Tat-Seng Chua. 2025b. [Personalized text generation with contrastive activation steering](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7128–7141, Vienna, Austria. Association for Computational Linguistics.
- Zhehao Zhang, Ryan A. Rossi, Branislav Kveton, Yijia Shao, Diyi Yang, Hamed Zamani, Franck Dernoncourt, Joe Barrow, Tong Yu, Sungchul Kim, Ruiyi Zhang, Jiuxiang Gu, Tyler Derr, Hongjie Chen, Junda Wu, Xiang Chen, Zichao Wang, Subrata Mitra, Nedim Lipka, and 2 others. 2025c. [Personalization of large language models: A survey](#). *Transactions on Machine Learning Research*. Survey Certification.

System Prompt

You are a strict data validation assistant. Your task is to compare an 'Introduction' and an 'Abstract' to determine if they describe the same specific subject matter, research topic, or underlying content.

Output Rules:

1. Return 'True' if the two texts describe the same core content or topic.
2. Return 'False' if the topics are unrelated, different, or contradictory.

Constraint: You must answer with EXACTLY one word: 'True' or 'False'. Do not provide explanations.

User Prompt

Please evaluate the following content pair:

Abstract
{abstract}

Introduction
{introduction}

Based on the content above, what is the determination (True/False)?

Figure 3: The prompt template used to query GPT-OSS-20B to classify the topic preservation of an abstract. {abstract} is replaced with the summary of the paper. {introduction} is replaced with the introduction the abstract is compared with.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging LLM-as-a-judge with MT-bench and chatbot arena](#). In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

A Prompt Templates

A.1 Topic Preservation Judge Prompt

The topic preservation prompt (Figure 3) instructs GPT-OSS-20B to compare a generated abstract with the original paper introduction and decide whether they describe the same document, returning a single binary label. GPT-OSS-20B decodes greedily with the default (medium) reasoning effort.

A.2 Preference Rate Judge Prompt

The quality evaluation prompt (Figure 4) presents GPT-OSS-20B with two abstracts written for the same paper and asks which is of higher qualitative standard, based on clarity, scientific rigor, and narrative flow. GPT-OSS-20B decodes greedily with the default (medium) reasoning effort. For topic preservation the prompt is capped at 3,500 tokens; when a sample exceeds this, only the introduction is truncated from the end, which preserves the abstract in full since abstracts contain at most 242 words.

A.3 Outline Creation Prompt

Papers exceeding the context window are split into chunks of up to 7,750 tokens with a 250-token overlap, processed separately, and merged in a second pass. For papers whose plain text exceeds the model’s context limit, each chunk is first processed individually using the prompt in Figure 5, which instructs the model to produce a compact bulleted outline of that chunk alone. These per-chunk outlines are then combined to form a single, structured outline using the prompt in Figure 6. For shorter papers that fit within the context window, the paper is directly summarized using the prompt in Figure 7 without chunking. In all cases, the model is a 4-bit quantized version of Qwen3-4B-Instruct-2507.

A.4 Abstract Generation Prompt

Two prompt templates are used for abstract generation, depending on whether the model receives author style examples. The zero-shot template (Figure 8) is used for the zero-shot baseline, the manual activation steering model, the steering vector prediction network and the hypernetwork model. It instructs the model to write a formal academic abstract from a structured paper outline, without any information about the target author. The few-shot template (Figure 9) is used for the few-shot baseline and the LoRA model. It extends the zero-shot template by providing three example abstracts by the target author, instructing the model to adopt that author’s style while remaining faithful to the outline content.

B Implementation Details

Environment All models are loaded with Unsloth and Transformers (Wolf et al., 2020), and k -NN retrieval uses FAISS (Douze et al., 2026). Every experiment runs on a single NVIDIA TITAN RTX

System Prompt

You are an expert peer reviewer for a top-tier scientific journal. Your task is to compare two abstracts (Abstract A and Abstract B) written for the same paper and determine which one is of higher qualitative standard.

CRITERIA FOR THE WINNER:

Choose the abstract that a mainstream scientific audience would find more compelling, professional, and publishable. Focus on:

1. Clarity & Precision: Which text explains the problem, methodology, and results more clearly and with better scientific vocabulary?
2. Impact & Rigor: Which abstract sounds more objective, authoritative, and convincing?
3. Flow: Which abstract has a better narrative arc that makes the reader want to read the full paper?

OUTPUT FORMAT:

Respond strictly in valid JSON format:

```
{
  "reasoning": "1-2 sentences explaining why
the winning abstract is more convincing
to a reviewer.",
  "winner": "<A, B>"
}
```

User Prompt

Please evaluate the following two abstracts.

ABSTRACT A:
{abstract_a}

ABSTRACT B:
{abstract_b}

System Prompt

You are a precise scientific summarizer.
Your goal: Synthesize the provided text chunk into a high-density, bulleted summary.

Constraints:

1. Max 400 words. Focus on maximum information density.
2. Preserve all key details important for writing an abstract or summary of the paper.
3. Remove redundancies and repetition.
4. Maximize information density: Prioritize factual content over linguistic fluency or reading flow.
5. Use a dry, compact, and technical style. Avoid flowery language or unnecessary transition words.
6. Especially do NOT use introductory filler (e.g. "This section discusses...").
7. Output ONLY the summary text.

User Prompt

Generate a comprehensive bulleted summary for this section:

{text_chunk}

Figure 4: The prompt template used to query GPT-OSS-20B for the quality win rate evaluation. {abstract_a} is replaced with the generated abstract and {abstract_b} with the human-written ground-truth abstract. Each pair is evaluated twice with the input order swapped to eliminate positional bias.

Figure 5: The prompt template used to query Qwen3-4B-Instruct-2507 4-bit for chunk summarizations. {text_chunk} is replaced with a chunk of the paper.

System Prompt

You are a senior scientific editor.
Your goal: Synthesize provided informations into a high-density, bulleted summary that contains every detail necessary to write a formal abstract.

Constraints:

1. Length: 400–500 words. Focus on maximum information density.
2. Structure: Use exactly four bulleted sections: CONTEXT (problem/objective), METHODOLOGY (design/sample/tools), RESULTS (specific findings/data), and CONCLUSION (significance/implications).
3. Requirements: DO NOT omit specific metrics, sample sizes, or p-values. Ensure the logical thread from hypothesis to implication is intact.
4. Use bullet points only.
5. Format: PLAIN TEXT ONLY. No Markdown headers (##). No intro/outro filler. Bold the section labels.

User Prompt

The following are summaries of a scientific paper's individual sections:

Section {1} Summary:
{summary_1}

Section {2} Summary:
{summary_2}

[...]

Section {x} Summary:
{summary_x}

Generate a comprehensive bulleted summary for these sections now. Ensure that no critical methodological detail or key finding is omitted, as this summary will serve as the sole basis for a formal abstract.

Figure 6: The prompt template used to query Qwen3-4B-Instruct-2507 4-bit for papers that exceed the context window and are first split into chunks. {summary_x} is replaced with the summary of chunk x . The [...] notation is a schematic abbreviation; the actual prompt contains one numbered section block per chunk.

System Prompt

You are a senior scientific editor.
Your goal: Synthesize provided informations into a high-density, bulleted summary that contains every detail necessary to write a formal abstract.

Constraints:

1. Length: 400–500 words. Focus on maximum information density.
2. Structure: Use exactly four bulleted sections: CONTEXT (problem/objective), METHODOLOGY (design/sample/tools), RESULTS (specific findings/data), and CONCLUSION (significance/implications).
3. Requirements: DO NOT omit specific metrics, sample sizes, or p-values. Ensure the logical thread from hypothesis to implication is intact.
4. Use bullet points only.
5. Format: PLAIN TEXT ONLY. No Markdown headers (##). No intro/outro filler. Bold the section labels.

User Prompt

The following is the paper you are required to summarize:

PAPER START:

{content}

Generate a comprehensive bulleted summary for these sections now. Ensure that no critical methodological detail or key finding is omitted, as this summary will serve as the sole basis for a formal abstract.

Figure 7: The prompt template used to query Qwen3-4B-Instruct-2507 4-bit for papers that fit within the model's context window. {content} is replaced with the plain text of the paper.

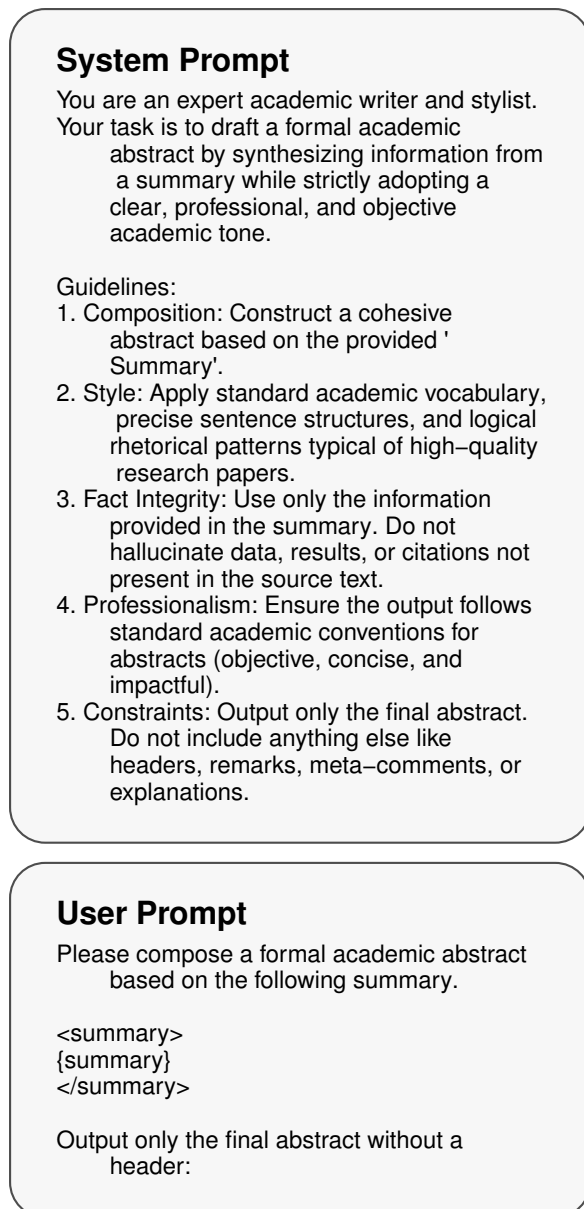


Figure 8: The prompt template used for zero-shot, manual steering, learned steering, and hypernetwork setup. {summary} is replaced with the outline of the paper.

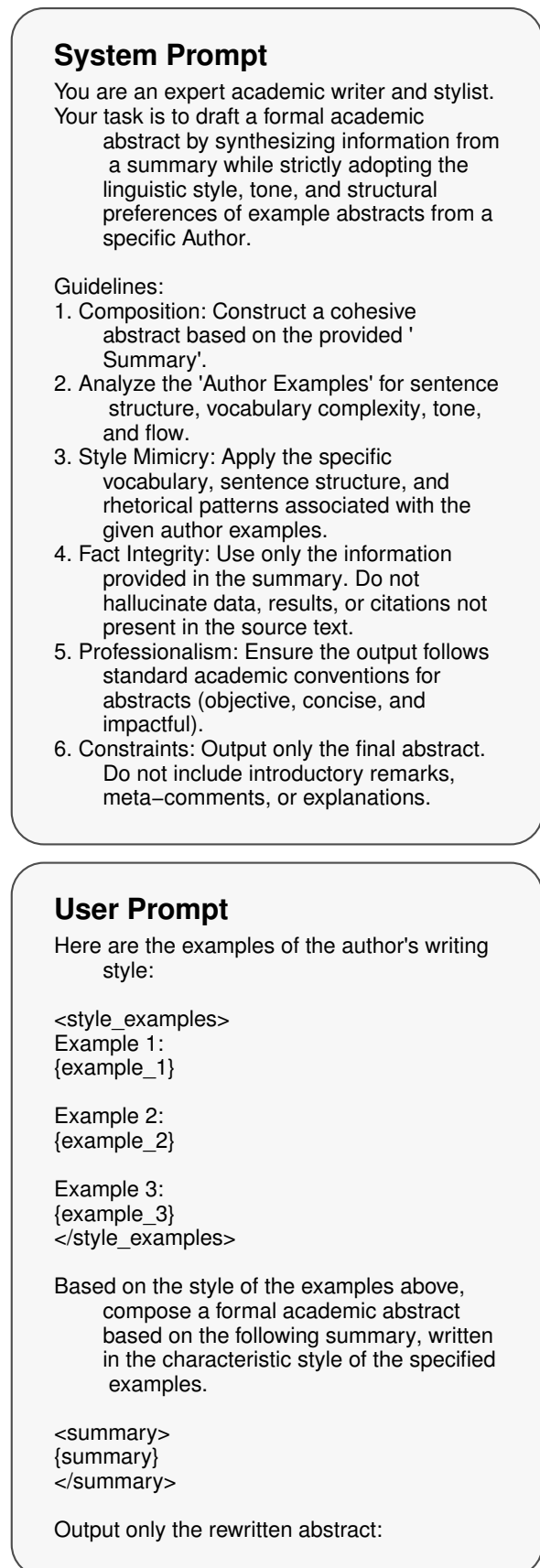


Figure 9: The prompt template used for the few-shot and LoRA. {summary} is replaced with the outline of the paper. {example_1}, {example_2} and {example_3} are replaced with the example abstracts of the author.

(24 GB VRAM) with an Intel Xeon Silver 4110 CPU and 376 GiB of RAM. Qwen3-4B-Instruct-2507 is loaded in full precision for generation; the judge is loaded in 4-bit. The longest prompt in the dataset is 1,851 tokens for outline-only inputs and 2,740 tokens with three in-context examples, both well within the context limit, so no input truncation occurs.

Author Classifier The task is multiclass classification from an abstract to its first author. We search over four encoders (bert-base-uncased, bert-base-multilingual-cased, distilbert-base-uncased, distilbert-base-multilingual-cased) and three modes: a classification head, k -NN on frozen embeddings, and k -NN on fine-tuned embeddings with the head removed. For the head mode we search six learning rates (1×10^{-6} to 5×10^{-5}), selecting by validation loss. For the retrieval modes we search two distance functions (L2, cosine) and $k \in \{1, 3, 5, 7, 9\}$, selecting by validation accuracy. Training runs for up to 100 epochs with batch size 16 and AdamW, evaluating each epoch, with early stopping after five epochs without improvement and restoration of the best checkpoint. All abstracts fit within the 512-token limit, so no truncation is applied. The selected configuration is distilbert-base-uncased, fine-tuned k -NN, $k = 3$, cosine distance, learning rate 2×10^{-5} . The k -NN database holds all training abstracts; for test-set evaluation the validation abstracts are added, and for the unseen-author set it holds the three reference abstracts per author.

LoRA Baseline Adapters are applied to q_proj and v_proj with $r = 8$ and $\alpha = 16$. We use AdamW with a learning rate of 2×10^{-4} , 5 warmup steps, weight decay 0.001, and NEFTune (Jain et al., 2024) with noise $\alpha = 5.0$. The effective batch size is 8 (per-device 2, 4 gradient accumulation steps), and sequences are padded or truncated to 4,096 tokens. Cross-entropy is computed on the abstract only, with the prompt masked. Training runs for at most 50 epochs, evaluating every 25 steps, with early stopping after 25 evaluations without validation-loss improvement and restoration of the best checkpoint.

Style Control Networks The author representation is produced by a frozen STAR encoder; a linear projection maps its 1,024-dimensional output to 128 dimensions before averaging over the

three references. The layer encoder maps the layer index to a 36-dimensional embedding followed by layer normalization; the module encoder (hypernetwork only) maps $\{q_proj, v_proj\}$ to a 32-dimensional embedding. Input dimensions are therefore $128 + 36 = 164$ for the steering network and $128 + 36 + 32 = 196$ for the hypernetwork. The backbone is a stack of four MLP blocks with a 512-dimensional up-projection, SiLU activations, and 5% dropout: the first block omits layer normalization, the second and third are residual blocks with pre-normalization, and the fourth normalizes without a residual connection. Backbone weights use Kaiming normal initialization with zero biases. The steering network has a single head projecting $512 \rightarrow 2,560$, zero-initialized in weights and bias. The hypernetwork has four heads emitting flattened adapters at $r = 8$, with shapes following Equation 4, i.e. $\mathbf{A} \in \mathbb{R}^{r \times d_{in}}$ and $\mathbf{B} \in \mathbb{R}^{d_{out} \times r}$: q_A ($8 \times 2,560$), q_B ($4,096 \times 8$), v_A ($8 \times 2,560$), and v_B ($1,024 \times 8$). The A -heads have zero weights and biases drawn uniformly from $[-1/\sqrt{d_{in}}, 1/\sqrt{d_{in}}]$; the B -heads are zero in both, matching standard LoRA initialization. Training follows the LoRA baseline’s configuration, with mixed precision (fp16) and gradient checkpointing added to reduce memory.

Listings 1–2 show the full architecture of the shared backbone used by both the activation vector prediction network and the hypernetwork model. The input module (Listing 1) encodes the author style embedding, the target layer index, and, for the hypernetwork, the target module. The backbone (Listing 3) consists of four sequential MLP blocks. The output heads (Listing 2) differ between the two models: the hypernetwork uses the four LoRA matrix heads, while the activation vector prediction network uses only the single steering vector head.

```

Input(
  (star_proj): Sequential(
    (0): Linear(in_features=1024,
out_features=128, bias=True)
    (1): LayerNorm((128,), eps=1e-05,
elementwise_affine=True)
  )
  (layer_depth_encoder): Sequential(
    (0): Embedding({num_of_layers}, 36)
    (1): LayerNorm((36,), eps=1e-05,
elementwise_affine=True)
  )
  (module_emb): Sequential(
    (0): Embedding(2, 32)
    (1): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
  )
)

```

Listing 1: Input to the style control networks. For the activation vector prediction network, the module embedding is removed. All inputs are concatenated to form the final input to the network. `num_of_layers` is replaced with the number of layers that the model is controlling.

```

Output(
  (final_proj_q_A): Linear(in_features
=512, out_features=20480, bias=True)
  (final_proj_q_B): Linear(in_features
=512, out_features=32768, bias=True)
  (final_proj_v_A): Linear(in_features
=512, out_features=20480, bias=True)
  (final_proj_v_B): Linear(in_features
=512, out_features=8192, bias=True)

  (final_steering): Linear(in_features
=512, out_features=2560, bias=True)
)

```

Listing 2: Output of the style control networks. Either the first four output heads are used for the LoRA hypernetwork or the `final_steering` output head is used for the activation vector prediction network. Each head emits a flattened matrix; the reshaped targets follow Equation 4.

```

ModuleDict(
  (mlp1): MLPResidualBlock(
    (mlp): Sequential(
      (0): Linear(in_features={
hidden_dim}, out_features=512, bias=
True)
      (1): SiLU()
      (2): Dropout(p=0.05, inplace=
False)
      (3): Linear(in_features=512,
out_features={hidden_dim}, bias=True)
    )
    (4): SiLU()
    (5): Dropout(p=0.05, inplace=
False)
  )
  (mlp2): MLPResidualBlock(
    (mlp): Sequential(
      (0): LayerNorm(({hidden_dim},),
eps=1e-05, elementwise_affine=True)
      (1): Linear(in_features={
hidden_dim}, out_features=512, bias=
True)
      (2): SiLU()
      (3): Dropout(p=0.05, inplace=False)
    )
    (4): Linear(in_features=512,
out_features={hidden_dim}, bias=True)
    (5): SiLU()
    (6): Dropout(p=0.05, inplace=False)
  )
  (mlp3): MLPResidualBlock(
    (mlp): Sequential(
      (0): LayerNorm(({hidden_dim},),
eps=1e-05, elementwise_affine=True)
      (1): Linear(in_features={
hidden_dim}, out_features=512, bias=
True)
      (2): SiLU()
      (3): Dropout(p=0.05, inplace=False)
    )
    (4): Linear(in_features=512,
out_features={hidden_dim}, bias=True)
    (5): SiLU()
    (6): Dropout(p=0.05, inplace=False)
  )
  (mlp4): Sequential(
    (0): LayerNorm(({hidden_dim},), eps
=1e-05, elementwise_affine=True)
    (1): Linear(in_features={hidden_dim
}, out_features=512, bias=True)
    (2): SiLU()
    (3): Dropout(p=0.05, inplace=False)
    (4): Linear(in_features=512,
out_features=512, bias=True)
    (5): SiLU()
  )
)

```

Listing 3: Backbone of the style control networks. The input is passed through the network in the order of the listing. `hidden_dim` is 196 for the LoRA hypernetwork and 164 for the activation prediction network.